1. LỊCH HỌC: MỘT TỐI + MỘT SÁNG

Mục tiêu: giải thích được bài, mở đúng hàm và demo được Nhập/Sửa/Xóa. Không cố học thuộc cả 29 trang. Bản này dùng source của repo QuanLyThietBi-midterm; tài liệu dài chỉ để tra khi vướng.

Tối nay — 2 giờ 30 phút

- **0–20 phút:** chạy ứng dụng; mở Program.cs, ThietBiService.cs và Repositories.cs. Tập nói bài toán và ba tầng theo trang 2.

- **20–50 phút:** học hai bảng, PK/FK, Code First và Migration. Vẽ quan hệ một–nhiều ra giấy, không nhìn tài liệu.

- **50–90 phút:** theo luồng Nhập ở trang 3. Chỉ đúng KiemTra, TonTai, Them và SaveChanges. Thử mã trùng, số lượng 0, ngày mai.

- **90–115 phút:** học Sửa/Xóa, cách chọn dòng và bảng trạng thái ở trang 4. Nghỉ ngắn giữa các chặng nếu cần.

- **115–150 phút:** đóng tài liệu, tự trả lời 15 câu trang 5. Câu nào vướng thì mở đúng hàm để kiểm tra, không đọc lại cả source.

Sáng mai — 1 giờ 30 phút

- **0–20 phút:** nói lại kiến trúc và luồng Nhập mà không nhìn tài liệu.

- **20–45 phút:** chạy demo trang 6: nhập TB006, sửa, thử lỗi, hủy xóa rồi xóa thật.

- **45–70 phút:** luyện 15 câu hỏi theo thứ tự ngẫu nhiên. Mỗi câu trả lời ngắn trước rồi chỉ code minh họa.

- **70–90 phút:** ôn những câu còn yếu; kiểm tra mở solution, SQL, Startup Project. Dừng học thêm chức năng mới.

Nếu thiếu thời gian: ưu tiên trang 2 → 3 → 4, sau đó các câu 1–10 ở trang 5. Async chi tiết, cấu hình nâng cao và Git là phần tra thêm, không phải trọng tâm đầu tiên.

Trước khi ngủ phải làm được: nói ba tầng; vẽ PK/FK; theo một lần Nhập; giải thích vì sao mã không sửa được. Sáng mai bổ sung demo và xử lý câu hỏi biến thể. Chốt giờ nghỉ để ngủ, không đổi toàn bộ thời gian ngủ thành đọc thuộc source.

2. NỀN TẢNG PHẢI NÓI ĐƯỢC

Bài toán và ba tầng

“Ứng dụng quản lý thiết bị phòng học theo loại, có Nhập/Sửa/Xóa và lưu SQL Server bằng Entity Framework. GUI nhận/hiện dữ liệu; BLL kiểm tra nghiệp vụ; DAL truy vấn và lưu database.”

```
FrmThietBi → ThietBiService → Repository → SQL Server
    GUI           BLL           DAL
```

Có bốn project nhưng ba tầng chức năng: **Entities** chứa model/DTO/interface dùng chung. **Program.Main** tạo repository, truyền vào service rồi chạy Form. GUI không tạo DbContext hoặc gọi SaveChanges; DAL không hiện MessageBox.

Hai bảng và quan hệ

- **LoaiThietBi:** MaLoai là PK, tối đa 10 ký tự; TenLoai bắt buộc, tối đa 100.

- **ThietBi:** MaTB là PK, tối đa 20; TenTB bắt buộc, tối đa 150; NgayMua, SoLuong, DonGia; MaLoai là FK bắt buộc.

- Một loại có nhiều thiết bị, mỗi thiết bị thuộc một loại. Ví dụ MANG có router và switch.

- Navigation: **LoaiThietBi.ThietBis** là tập thiết bị; **ThietBi.LoaiThietBi** là loại của thiết bị.

EF và Migration — chỉ cần chắc những ý này

- **DbContext:** quản lý phiên làm việc, model và theo dõi thay đổi. **DbSet:** tập entity để truy vấn/ghi.

- **Code First:** định nghĩa class C# → Migration → tạo/cập nhật schema SQL. Không lấy database để sinh entity như Database First.

- **Up/Down:** tạo/thay đổi và đảo thay đổi schema. **Seed:** thêm dữ liệu mẫu. **__MigrationHistory:** theo dõi migration đã áp dụng.

- **Add chưa lưu; SaveChanges mới ghi.** Dữ liệu còn khi đóng ứng dụng vì nằm trong SQL, không chỉ trong lưới.

- **decimal(18,2):** tổng 18 chữ số, 2 số lẻ. Dùng decimal cho tiền. Annotation Column kết hợp HasPrecision(18,2) và Migration để chốt kiểu SQL.

Bốn annotation: Key = khóa; StringLength = giới hạn chuỗi; Required = bắt buộc; ForeignKey = nối FK/navigation. Required không chặn số lượng âm: quy tắc > 0 nằm trong BLL.

Mở ngay: Entities/ThietBi.cs → DAL/ThietBiContext.cs → DAL/Migrations/Configuration.cs. Tập chỉ PK, FK và hai DbSet.

3. NHẬP MỚI: PHẦN CẦN HỌC KỸ NHẤT

Nhớ đường đi và mở được đúng code

```
btnNhap.Click → LuuAsync(false) → DocDuLieu()
→ ThucHienGhiAsync → service.Nhap(input)
→ KiemTra → TonTai → Them → SaveChanges
→ LayDanhSach → GanDanhSach → LamMoi
```

GUI/FrmThietBi.cs: DocDuLieu lấy các ô nhập thành ThietBiInput. SoLuong và DonGia vẫn là string để BLL kiểm tra được cả dữ liệu sai định dạng.

BLL/ThietBiService.cs: Nhap kiểm tra rồi gọi repository. Phần cốt lõi:

```
var entity = KiemTra(input);
if (thietBi.TonTai(entity.MaTB))
    throw new BusinessException("Mã thi□t bị đã t□n tại.");
thietBi.Them(entity);
```

KiemTra làm gì? Nói theo thứ tự

1. Chặn input null; Trim mã, tên, mã loại. Chuỗi null được thay bằng chuỗi rỗng qua toán tử ??.

2. Mã/tên không rỗng; đúng độ dài 20/150; phải chọn loại đang tồn tại.

3. Ngày mua không sau hôm nay; không trước 01/01/1753 vì SQL datetime.

4. TryParse số lượng thành int, bắt buộc > 0. TryParse sai thì báo lỗi nghiệp vụ thay vì Parse gây FormatException.

5. TryParse giá decimal theo vi-VN, > 0, không vượt decimal(18,2), không có giá trị lẻ vượt 2 chữ số.

6. Tạo entity có kiểu số đúng. Nhap kiểm tra trùng mã sau chuẩn hóa rồi mới gọi Them.

Ví dụ cần nói: “ TB006 ” thành “TB006”; nếu đã tồn tại thì không gọi Them. Ngày mai, số lượng 0 hoặc giá abc đều dừng ở BLL.

DAL/Repositories.cs: lưu thực tế

```
using (var db = new ThietBiContext())
{
    db.ThietBis.Add(entity);
    db.SaveChanges();
}
```

using bảo đảm Dispose context khi xong hoặc có lỗi. Nếu lưu lỗi, ngoại lệ truyền về GUI; không báo thành công. Sau ghi, GUI truy vấn lại để lưới phản ánh database.

Câu nâng cao: TonTai không thay thế PK SQL. Hai phiên có thể cùng thấy mã chưa tồn tại; database vẫn phải chặn mã trùng.

4. SỬA, XÓA VÀ TRẠNG THÁI GIAO DIỆN

Sửa — bảo vệ khóa chính ở ba nơi

1. **ChonDong:** lấy ThietBiRow, giữ maDangChon, nạp ô nhập; txtMaTB.ReadOnly=true.

2. **BLL.Sua:** so maGoc với input đã chuẩn hóa; nếu đổi mã thì từ chối.

3. **DAL.Sua:** Find(maGoc), kiểm tra null, chỉ gán TenTB/NgayMua/SoLuong/DonGia/MaLoai rồi SaveChanges. Không gán MaTB, không Add dòng mới.

Xóa — xác nhận trước

XoaAsync hỏi Yes/No. Không Yes thì return trước khi gọi service. Yes → BLL kiểm tra mã/tồn tại → DAL Find → Remove → SaveChanges → tải lại. Chỉ xóa dòng trên DataGridView chưa đồng nghĩa xóa SQL.

Ghi nhớ trạng thái

Trạng thái	Nhập	Sửa/Xóa	Ô mã
Mới, đã sẵn sàng	Bật	Tắt	Sửa được
Chọn dòng	Tắt	Bật	Khóa
Đang xử lý/chưa nạp được	Tắt	Tắt	Vùng nhập tắt

- **maDangChon:** biết đang nhập mới hay sửa dòng nào.

- **sanSang:** nạp dữ liệu được để thao tác. **dangXuLy:** đang nạp/ghi, chặn lệnh cạnh tranh.

- **dangGanDuLieu:** bỏ qua SelectionChanged phát sinh khi bind/reset bằng code.

- **LamMoi:** chỉ xóa ô nhập/lựa chọn; không xóa SQL. Ngày hôm nay, loại chưa chọn, focus mã.

ComboBox: DisplayMember=TenLoai là chữ hiển thị; ValueMember=MaLoai là FK dùng lưu. Chọn dòng bằng SelectedValue=row.MaLoai. Lưới hiện TenLoai nhờ DAL chiếu sang ThietBiRow.

Enter/Esc: AcceptButton là Nhập; CancelButton là Thoát. Nút Thoát gọi Close; duy nhất FormClosing hỏi xác nhận. No đặt e.Cancel=true, tránh hỏi hai lần.

Async và lỗi — câu trả lời đủ dùng

SQL đồng bộ chạy trong Task.Run để UI không bị chặn. Chụp input trên UI trước; await xong mới gán control. Không dùng chung DbContext giữa các thread. Timer chỉ hiển thị thời gian chờ, không phải hủy SQL.

daLuu: phân biệt ghi lỗi với đã ghi nhưng tải lại lỗi. Trường hợp đã ghi không hướng dẫn người dùng nhập lặp. finally dọn cờ, dừng timer và cập nhật nút.

5. 15 CÂU VẤN ĐÁP ƯU TIÊN

1. Bài em có những tầng nào? GUI nhận/hiện dữ liệu, BLL xử lý quy tắc, DAL dùng EF truy cập SQL. Entities là model/hợp đồng chung.

2. Nhấn Nhập thì chạy qua đâu? LuuAsync → DocDuLieu → service.Nhap/KiemTra → repository.Them → SaveChanges → tải lại lưới.

3. BLL kiểm tra gì? Rỗng/độ dài, loại tồn tại, ngày không tương lai, số lượng nguyên > 0, giá decimal > 0 và giới hạn, mã trùng khi nhập.

4. Vì sao không kiểm tra mỗi ở Form? GUI chỉ là một đường nhập; BLL phải bảo vệ nghiệp vụ cả khi nguồn gọi khác truyền dữ liệu vào.

5. Code First là gì? Dùng entity C# và cấu hình model, tạo/cập nhật schema bằng Migration; không sinh entity từ database có sẵn.

6. Migration khác Seed? Migration đổi schema qua Up/Down; Seed thêm dữ liệu. Thêm loại MANG không phải thêm cột nên không cần migration mới.

7. PK/FK và một–nhiều của bài? MaTB định danh thiết bị; MaLoai ở ThietBi trỏ sang LoaiThietBi; một loại có nhiều thiết bị.

8. Add với SaveChanges khác nhau? Add đưa entity vào context để thêm; SaveChanges mới gửi thay đổi xuống SQL. Remove cũng cần SaveChanges.

9. Vì sao giá dùng decimal? Phù hợp tiền, tránh sai số số thực nhị phân kiểu double; khớp SQL decimal(18,2), tức 16 số nguyên và 2 số lẻ.

10. Làm sao không cho đổi mã khi sửa? Khóa ô mã ở GUI, BLL đối chiếu maGoc, DAL Find theo mã gốc và không gán lại PK.

11. Vì sao phải Trim trước khi kiểm tra trùng? “ TB006 ” và “TB006” phải được xử lý thành cùng mã. PK SQL vẫn bảo vệ trường hợp hai phiên thêm đồng thời.

12. DisplayMember/ValueMember là gì? Tên loại để hiện và mã loại để lưu. Chọn dòng nạp ComboBox theo FK, không theo vị trí dòng.

13. Vì sao thoát không hỏi hai lần? Các cách đóng đi qua một FormClosing. Nút chỉ gọi Close; No đặt e.Cancel=true.

14. Vì sao chạy Task.Run? EF trong repository đang đồng bộ; chuyển việc chờ SQL khỏi UI. await xong quay lại UI để bind, không dùng chung context giữa thread.

15. Làm sao biết ứng dụng lưu được thật? Nhập/sửa/xóa rồi đóng và mở lại để đối chiếu SQL. Test repository giả chỉ kiểm tra BLL, không chứng minh SQL thật đã chạy.

Cách trả lời câu lạ: xác định thuộc UI/BLL/DAL/database → nêu luồng → mở hàm → đưa một ví dụ. Không đoán là có tính năng chưa làm. Bản này chưa có đăng nhập, tìm kiếm hay rowversion chống ghi đè khi sửa đồng thời.

6. DEMO SÁNG MAI VÀ TỜ NHẮC TRƯỚC THI

Demo một vòng — làm trên bản sao dữ liệu nếu cần

1. Chạy GUI; chọn TB001 để thấy các ô nạp đúng và mã bị khóa.

2. Bấm Mới: ô sạch, loại chưa chọn, ngày hôm nay, Nhập bật.

3. Nhập TB006 (nếu chưa có), Router thử nghiệm, loại MANG, hôm nay, SL=2, giá=1200000. Lưu và nhìn lưới.

4. Nhập lại " TB006 ": phải báo trùng. Thử ngày mai, SL=0, giá=abc: phải báo lỗi. Sau mỗi lần sửa lại dữ liệu hợp lệ.

5. Chọn TB006, Sửa số lượng thành 7. Giải thích không tăng số dòng vì Find và cập nhật entity cũ.

6. Đóng/mở lại: TB006 vẫn còn. Xóa chọn No thì giữ nguyên; chọn Yes thì mất. Đóng/mở lại để kiểm tra đã xóa.

7. Bấm X/Esc, chọn No: Form còn, chỉ hỏi một lần. Không sửa schema hoặc xóa database trong buổi demo.

Chỉ cần tìm nhanh sáu nơi

- **GUI/Program.cs:** Main, nơi tạo service/repository và Application.Run.

- **Entities/ThietBi.cs:** annotation, PK/FK, navigation.

- **DAL/ThietBiContext.cs:** DbSet, HasPrecision và quan hệ.

- **BLL/ThietBiService.cs:** KiemTra, Nhap, Sua, Xoa.

- **DAL/Repositories.cs:** LayTatCa, Find, Add/Remove, SaveChanges.

- **GUI/FrmThietBi.cs:** ChonDong, LamMoi, LuuAsync, XoaAsync, ThucHienGhiAsync.

Hai chỗ dễ bị hỏi thêm

Có MANG nhưng Wi-Fi vẫn là máy tính? Source được đối chiếu đã có MANG, nhưng mẫu TB003 vẫn gán MAYTINH. Thêm loại không tự đổi FK của dòng đã có. Chọn TB003 → MANG → Sửa cho database hiện tại; sửa mẫu để database tạo mới cũng đúng. Không cần Migration.

Nếu thêm cột phòng học? Sửa entity → DTO → BLL → DAL → GUI → Migration. Phải quyết định giá trị cho dòng cũ nếu cột bắt buộc. Chỉ thêm TextBox là chưa đủ.

Khi gặp lỗi lúc mở

- Chạy nhầm Class Library: đặt QuanLyThietBi.GUI làm Startup Project.

- Project incompatible: kiểm tra workload .NET desktop development/targeting pack.

- NETSDK1004: Restore NuGet. SQL lỗi: xem máy chủ/LocalDB và Cấu hình CSDL.

- Đổi tên sinh viên: GUI/App.config, HoTenSinhVien. Không đổi entity vì tiêu đề.

Trước lúc đi thi: mở được solution và Form; SQL hoạt động; biết vị trí sáu file trên. Đừng tải bản khác hoặc thay nhiều code sát giờ thi. Khi trả lời, nói ý chính trước, chỉ code sau, demo cuối cùng.

Nguồn đối chiếu: repo QuanLyThietBi-midterm, code tại commit 3cfa383bd16d27ebd2ac768c22ba30846c4b7bcc. Đây là bản rút gọn để ôn cấp tốc, không phải cam kết bao phủ mọi câu hỏi.